

Capitolo 3

Modello dei dati e basi operative

Nel capitolo precedente abbiamo chiarito che cosa significa fare query in MongoDB e abbiamo distinto le operazioni CRUD dalle aggregazioni. Adesso facciamo un passo molto importante: prima di interrogare bene i dati, dobbiamo capire bene come sono fatti.

Questa osservazione può sembrare banale, ma in pratica è decisiva. Molti errori nelle query non nascono da una sintassi sbagliata; nascono dal fatto che chi scrive la query non ha ancora capito bene la forma del dato. MongoDB memorizza i record come documenti BSON, raggruppati in collection, a loro volta contenute in un database. Inoltre i documenti possono contenere non solo campi semplici, ma anche documenti annidati, array e array di documenti.

([mongodb.com])(https://www.mongodb.com/docs/manual/core/databases-and-collections/?utm_source=chatgpt.com))

In questo capitolo lavoreremo proprio su questa base. Non faremo ancora query complesse, ma costruiremo il modo giusto di guardare il database.

3.1 Database, collection e documenti

MongoDB organizza i dati in tre livelli principali:

- il database
- la collection
- il documento

Un database contiene una o più collection. Una collection contiene uno o più documenti. MongoDB descrive proprio così il proprio modello: i record sono documenti BSON, raccolti in collection, e i database contengono una o più collection.

([mongodb.com])(https://www.mongodb.com/docs/manual/core/databases-and-collections/?utm_source=chatgpt.com))

Se arrivi dal mondo SQL, puoi usare questa analogia iniziale:

- database → database
- tabella → collection
- riga → documento
- colonna → campo

È un'analogia utile, ma non perfetta. La vera differenza è che un documento MongoDB è molto più ricco di una classica riga tabellare. Una riga SQL, di solito, è piatta: una serie di colonne. Un documento MongoDB, invece, può contenere altri oggetti al suo interno.

Facciamo subito un esempio concettuale.

Immagina di dover rappresentare un cliente. In un sistema relazionale potresti avere:

- tabella clienti
- tabella indirizzi cliente
- tabella contatti cliente

In MongoDB, in molti casi, puoi avere un singolo documento cliente che contiene direttamente i suoi indirizzi e i suoi contatti. Questo non significa che si faccia sempre così, ma significa che il modello documentale lo rende naturale. MongoDB supporta infatti relazioni rappresentate tramite documenti incorporati, array e array di documenti.

([mongodb.com])(https://www.mongodb.com/docs/manual/data-modeling/?utm_source=chatgpt.com))

Nel nostro database aziendale, per esempio, il database è company_demo. Al suo interno troviamo collection come:

- clients
- suppliers
- products
- salesOrders
- purchaseOrders
- warehouses
- inventoryMovements

- employees

Questo già ci dice una cosa importante: il nostro sistema non vive in una sola collection monolitica. È un piccolo ecosistema di collections collegate logicamente tra loro.

3.2 Il documento: l'unità fondamentale di lavoro

In MongoDB, il documento è l'unità base di memorizzazione. La documentazione ufficiale chiarisce che MongoDB salva i record come documenti BSON, cioè una rappresentazione binaria del JSON con tipi aggiuntivi rispetto al JSON puro.

([mongodb.com](https://www.mongodb.com/docs/manual/core/document/?utm_source=chatgpt.com))

Questo punto va capito bene.

Quando lavori con MongoDB, nella maggior parte dei casi stai pensando in termini di documenti.

Leggi documenti.

Aggiorni documenti.

Inserisci documenti.

Cancelli documenti.

Aggregi partendo da documenti.

Un documento può essere semplice oppure ricco.

Per esempio, un prodotto potrebbe avere:

- codice
- nome
- categoria
- prezzo
- costo
- marca

Ma può avere anche:

- un oggetto attributes
- un array tags
- un riferimento a un fornitore
- un riferimento al magazzino preferito

Questo rende MongoDB molto espressivo. Allo stesso tempo, però, ti obbliga a osservare bene la forma del documento prima di scrivere una query.

Un altro dettaglio importante è che il documento non è solo “contenuto”. È anche il perimetro naturale di molte operazioni atomiche. Se modifichi un singolo documento, MongoDB tratta quella scrittura in modo atomico a livello di documento. Questo significa che il documento non è solo un contenitore logico, ma anche una specie di unità operativa molto importante.

3.3 BSON: perché MongoDB non usa JSON puro

Spesso si dice che MongoDB memorizza dati in JSON. In realtà è una semplificazione comoda, ma non del tutto corretta. MongoDB memorizza documenti in BSON, che è una rappresentazione binaria del JSON con tipi aggiuntivi. Tra questi tipi ci sono per esempio date, ObjectId, numeri di diversi formati e dati binari.

([mongodb.com])(https://www.mongodb.com/docs/manual/core/document/?utm_source=chatgpt.com)

Perché questa cosa è importante per noi?

Perché spiega alcune situazioni che, altrimenti, sembrerebbero un po' misteriose.

Per esempio:

- il campo `_id` non è una semplice stringa casuale, ma spesso è un ObjectId
- una data non è trattata come testo generico
- alcuni confronti dipendono anche dal tipo dei valori

Facciamo un esempio molto semplice.

Se memorizzi la data di un ordine come vero valore data, il database può trattarla come data: ordinare correttamente, confrontare intervalli temporali, fare filtri per periodo.

Se invece quella data fosse solo una stringa scritta male o in formato incoerente, molte operazioni diventerebbero più fragili.

Quindi il punto non è solo teorico. Capire che MongoDB usa BSON ti aiuta a capire perché conviene progettare bene i campi.

3.4 Il ruolo speciale del campo `_id`

Ogni documento MongoDB ha un campo speciale chiamato `_id`. Anche quando non lo specifichi tu, MongoDB lo crea. Nella pratica, questo campo identifica in modo univoco il documento all'interno della collection.

È utile vederlo come il “document key” naturale del record.

Nel nostro database aziendale, per esempio, un cliente può avere un campo code leggibile, come CLI0001, utile a livello business. Ma il vero identificatore tecnico del documento è `_id`.

Questa distinzione è molto comune:

- `_id` serve al database come identificatore interno univoco
- campi come code, sku o orderNumber servono al dominio applicativo

Un esempio chiarisce bene il concetto.

Per un ordine cliente, il business ragiona su qualcosa come:

SO-2025-000145

È un numero d'ordine leggibile e comodo per utenti e ufficio commerciale.

MongoDB, però, gestisce anche il proprio identificatore `_id`, che viene usato molto spesso come chiave tecnica, anche per collegare documenti di collections diverse.

Più avanti vedremo questo meccanismo soprattutto nei capitoli su `$lookup` e sulle relazioni tra collections.

3.5 Campi semplici, campi annidati e array

Qui iniziamo a entrare davvero nel cuore del modello documentale.

Un documento MongoDB può contenere:

- campi semplici
- documenti annidati
- array
- array di documenti

MongoDB lo supporta in modo nativo: il valore di un campo può essere qualsiasi tipo BSON valido, inclusi documenti, array e array di documenti.

([mongodb.com])(https://www.mongodb.com/docs/manual/data-modeling/?utm_source=chatgpt.com))

Campi semplici

Sono i campi più immediati: stringhe, numeri, booleani, date.

Esempi dal nostro dominio aziendale:

- `companyName`
- `status`

- creditLimit
- unitPrice
- orderDate

Su questi campi, normalmente, le query sono le più facili da immaginare.

Per esempio:

- clienti con status = ACTIVE
- prodotti con unitPrice > 100
- ordini creati dopo una certa data

Documenti annidati

Un documento annidato è un oggetto dentro un oggetto.

Per esempio, un prodotto può avere un campo attributes che contiene al suo interno altri campi, come colore, peso e dimensioni.

Concettualmente:

- documento prodotto
 - name
 - category
 - attributes
 - color
 - weightKg
 - dimensionsCm

Questo ti permette di tenere insieme informazioni fortemente correlate.

Array

Un array è un elenco di valori.

Per esempio, un prodotto può avere un array di tag:

- ["wireless", "office", "bestseller"]

Oppure un cliente può avere una serie di etichette commerciali:

- ["vip", "nord", "retail"]

Array di documenti

Questa è una delle forme più interessanti del modello MongoDB.

Un cliente può avere un array contacts, dove ogni elemento non è una semplice stringa, ma un piccolo documento con nome, ruolo, email e telefono.

Lo stesso vale per un ordine, che può avere un array items. Ogni riga ordine è un documento dentro l'array.

E qui si capisce bene perché MongoDB sia così adatto a dati gerarchici: un ordine non è più una riga piatta, ma un oggetto che contiene davvero le sue righe.

3.6 Il database di esempio visto con occhi tecnici

Nel capitolo 1 abbiamo presentato il database aziendale in modo intuitivo. Adesso possiamo guardarlo con occhi un po' più tecnici.

Vediamo le collection principali una per una.

clients

Contiene i clienti dell'azienda.

Ha campi semplici come codice, nome, stato, email e condizioni commerciali.

Ha anche array come addresses, contacts e tags.

Questa collection è ottima per esempi su:

- query semplici
- campi annidati
- array di documenti
- update su array
- proiezioni

suppliers

Contiene i fornitori.

Ha campi descrittivi, indicatori come rating, tempi di consegna e categorie merceologiche servite.

È utile per:

- query filtrate
- confronti tra fornitori
- join logici con products
- aggregazioni per paese o categoria

products

È il catalogo prodotti.

Qui troviamo campi semplici come SKU, nome, categoria, prezzi, stato.

Troviamo anche campi annidati come attributes, array come tags, e riferimenti verso suppliers e warehouses.

Questa collection è perfetta per quasi tutto:

- query base
- confronti numerici
- filtri su nested fields
- ordinamenti
- aggregazioni per categoria e brand

salesOrders

È una delle collection più importanti del libro.

Contiene gli ordini cliente.

Oltre ai campi di testata, contiene un array items, cioè le righe ordine.

Qui entrano in gioco concetti fondamentali come:

- array di documenti
- unwind
- group
- report per cliente, mese, prodotto o canale

purchaseOrders

È simile a salesOrders, ma dal lato acquisti.

È utile per confrontare dinamiche di approvvigionamento e vendita.

warehouses

Rappresenta i magazzini fisici.

È una collection più semplice, ma molto utile per join e aggregazioni logistiche.

inventoryMovements

Questa collection è preziosissima per le analisi.

Ogni documento rappresenta un movimento di magazzino: ingresso, uscita, trasferimento o rettifica.

È perfetta per:

- query temporali
- aggregazioni per prodotto
- aggregazioni per magazzino
- esempi con window functions più avanti nel libro

employees

Contiene i dipendenti.

È utile per collegare ordini, magazzini e ruoli interni.

In pratica, il database è stato progettato per avere sia collections abbastanza semplici, sia collections ricche e annidate. Questo equilibrio è voluto: serve a farci lavorare bene sia sui capitoli introduttivi, sia su quelli avanzati.

3.7 Embedding e riferimenti: due modi di modellare le relazioni

Quando modelli dati in MongoDB, una delle prime domande che devi farti è:

“Questa informazione la tengo dentro lo stesso documento oppure la metto in un’altra collection?”

In pratica, hai due strade principali:

- embedding, cioè incorporare i dati nello stesso documento

- reference, cioè tenere un riferimento a un altro documento in un'altra collection

MongoDB supporta entrambe le strategie nel proprio modello dati. I documenti possono incorporare altri documenti e array, ma possono anche rappresentare relazioni tramite riferimenti tra collections. ([mongodb.com](https://www.mongodb.com/docs/manual/data-modeling/?utm_source=chatgpt.com))

Esempio di embedding

In salesOrders, le righe ordine stanno dentro il documento dell'ordine, nell'array items.

Perché è una buona scelta?

Perché le righe appartengono naturalmente all'ordine. Quando pensi a un ordine, pensi già alle sue righe. Le due cose stanno bene insieme.

Esempio di riferimento

In products, invece, il fornitore non viene copiato integralmente dentro ogni prodotto.

Si usa un riferimento, per esempio supplierId, verso la collection suppliers.

Perché?

Perché un fornitore è un'entità autonoma, con vita propria, che può essere collegata a molti prodotti. Duplicarlo completamente in ogni prodotto sarebbe spesso scomodo e poco elegante.

Questo è un punto didattico molto importante: il nostro database mostra entrambe le strategie, così potremo usare esempi realistici sia per le query su documenti incorporati sia per i capitoli su più collections.

3.8 Dot notation: il modo naturale di navigare nei documenti

MongoDB usa la dot notation per accedere ai campi di documenti annidati e agli elementi degli array. La documentazione lo dichiara esplicitamente: la dot notation serve per accedere ai campi di un embedded document e agli elementi di un array.

([mongodb.com])(https://www.mongodb.com/docs/v8.0/core/document/?utm_source=chatgpt.com)

Questa è una delle idee più importanti da fissare presto, perché la useremo continuamente.

Supponiamo che un cliente abbia un indirizzo di spedizione con un campo city.

Per riferirci a quella città dentro una query, useremo qualcosa come:

```
"addresses.city"
```

Oppure, se un prodotto ha un oggetto attributes con un campo color, potremo pensare a qualcosa come:

```
"attributes.color"
```

L'idea è semplice: invece di fermarti al primo livello del documento, entri nei livelli successivi usando il punto.

MongoDB specifica anche che per accedere a un elemento specifico di un array si può usare la notazione con indice zero-based, per esempio "contribs.2" per il terzo elemento.

([mongodb.com])(https://www.mongodb.com/docs/v8.0/core/document/?utm_source=chatgpt.com)

Nel nostro libro useremo la dot notation per:

- filtrare campi annidati
- proiettare campi annidati
- aggiornare campi annidati
- lavorare con array e array di documenti

È uno di quegli strumenti piccoli ma fondamentali: una volta interiorizzato, ti sembrerà naturale.

3.9 Esempi iniziali per leggere la struttura dei dati

Anche se i capitoli sulle query vere inizieranno poco dopo, qui possiamo già fare alcuni piccoli esercizi esplorativi. Non servono ancora a mostrare la piena potenza del linguaggio, ma sono perfetti per “leggere” il database.

Vedere alcuni clienti

```
db.clients.find().limit(2)
```

Che cosa fa?

Mostra due documenti della collection clients.

Perché è utile?

Perché ti abitua a guardare la forma reale dei dati, non solo la loro descrizione teorica.

Vedere alcuni prodotti

```
db.products.find().limit(2)
```

Qui puoi già osservare se ci sono campi semplici, oggetti annidati, tag, riferimenti.

Vedere un ordine cliente

```
db.salesOrders.find().limit(1)
```

Questo esempio è molto istruttivo, perché quasi sicuramente ti mostrerà un documento con una struttura più ricca, compreso l'array items.

Se fai questi tre piccoli test in mongosh, inizi a vedere una cosa importante: non tutte le collection hanno la stessa “profondità” strutturale. Alcune sono più semplici, altre sono più gerarchiche.

Questa osservazione ti tornerà utilissima più avanti.

3.10 Leggere prima la forma, poi il contenuto

Questa è una regola pratica che ti consiglio davvero di adottare da subito.

Quando incontri una nuova collection, non partire immediatamente con una query complessa. Fai prima una piccola esplorazione.

Per esempio:

1. guarda qualche documento
2. individua i campi principali
3. verifica se ci sono oggetti annidati
4. verifica se ci sono array
5. capisci quali campi sembrano identificativi
6. prova a immaginare quali query avranno più senso

È un po' come entrare in una casa nuova. Prima guardi com'è fatta. Solo dopo inizi a cercare cose specifiche.

Nel nostro database, per esempio, questa strategia evita molti errori.

Se sai che salesOrders contiene un array items, non proverai a trattare le righe ordine come se fossero campi piatti.

Se sai che products ha un oggetto attributes, non cercherai un campo inesistente al primo livello.

Se sai che clients contiene contacts come array di documenti, capirai subito che alcune query richiederanno una logica un po' più attenta.

3.11 Convenzioni usate nel libro

Da questo capitolo in avanti, il libro seguirà alcune convenzioni pratiche, così gli esempi restano coerenti.

Useremo sempre il database di esempio

Quando scriveremo query, assumeremo quasi sempre di essere dentro company_demo.

Useremo mongosh

Gli esempi saranno scritti soprattutto in stile mongosh, perché è il modo più diretto per imparare il linguaggio.

Useremo nomi di collection realistici

Lavoreremo su collection come clients, products, salesOrders e così via, non su esempi astratti troppo artificiali.

Procederemo per complessità crescente

Prima query semplici.

Poi filtri più articolati.

Poi nested fields e array.

Poi update più sofisticati.

Poi aggregazioni.

Questa progressione è intenzionale: MongoDB può essere molto potente, ma il modo giusto di impararlo è per strati, non cercando di fare tutto insieme dal primo minuto.

3.12 Un primo esercizio mentale: come immaginare una query corretta

Prima ancora di scrivere sintassi, conviene allenare un piccolo schema mentale.

Quando devi interrogare una collection, chiediti sempre:

- qual è la collection giusta?
- il dato che cerco è in un campo semplice o annidato?
- sto cercando documenti o un risultato riassuntivo?
- la struttura include array?
- mi basta find() oppure sto entrando nel territorio di aggregate()?

Facciamo un esempio.

Domanda: “Voglio vedere i prodotti attivi della categoria ACCESSORIES.”

Ragionamento:

- collection giusta: products
- dato cercato: campi semplici
- voglio documenti, non un riepilogo
- non mi serve aggregare
- quindi sono nel territorio naturale di find()

Adesso un secondo esempio.

Domanda: “Voglio sapere quante righe ordine abbiamo venduto per categoria prodotto.”

Qui il ragionamento cambia:

- collection di partenza: probabilmente salesOrders
- il dato interessante sta nelle righe items
- non voglio singoli ordini, voglio una sintesi
- quindi sto entrando nel territorio di aggregate()

Vedi la differenza?

La qualità della query dipende moltissimo dalla qualità del ragionamento iniziale.

3.13 Il modello dati come alleato delle query

Una cosa che voglio sottolineare con forza è questa: in MongoDB il modello dati e il modo di interrogare i dati sono molto legati.

In un database relazionale, spesso pensi prima alle tabelle e poi alle query.

In MongoDB questa connessione è ancora più stretta, perché il modo in cui hai modellato documenti, array e riferimenti influenza direttamente il modo in cui scriverai find(), update() e aggregate().

Per esempio:

- se incorpori le righe ordine in salesOrders, renderai molto naturale leggere un ordine completo
- se separi i fornitori in una collection autonoma e li colleghi con supplierId, renderai naturale usare join logici e \$lookup
- se metti attributi specifici dentro un oggetto attributes, userai spesso dot notation
- se usi array di documenti, dovrai ragionare bene su filtri e update sugli array

Quindi, quando studiamo le query, in realtà stiamo anche studiando il modello dati. E viceversa.

3.14 Conclusione del capitolo

In questo capitolo abbiamo costruito la base strutturale che ci servirà per tutto il resto del libro.

Abbiamo visto che MongoDB organizza i dati in database, collection e documenti, e che i documenti sono BSON document, non semplici righe piatte. Inoltre, i documenti possono contenere campi semplici, documenti annidati, array e array di documenti, e MongoDB usa la dot notation per accedere a campi nested ed elementi di array.

([mongodb.com](https://www.mongodb.com/docs/manual/core/databases-and-collections/?utm_source=chatgpt.com))

Abbiamo anche letto il nostro database aziendale con un taglio più tecnico, distinguendo le collections più semplici da quelle più ricche e mostrando come il modello combini embedding e riferimenti. Questa scelta è intenzionale: ci permetterà di studiare in modo realistico sia query semplici sia casi più avanzati su più collections.

([mongodb.com](https://www.mongodb.com/docs/manual/data-modeling/?utm_source=chatgpt.com))

La lezione più importante del capitolo, però, è forse questa: prima di scrivere una query, devi capire la forma del dato.

Capire dove stanno i campi, se ci sono oggetti annidati, se ci sono array e quale collection contiene davvero l'informazione che stai cercando.

Nel prossimo capitolo entreremo finalmente nelle prime query operative di lettura, iniziando da `find()` e `findOne()`.